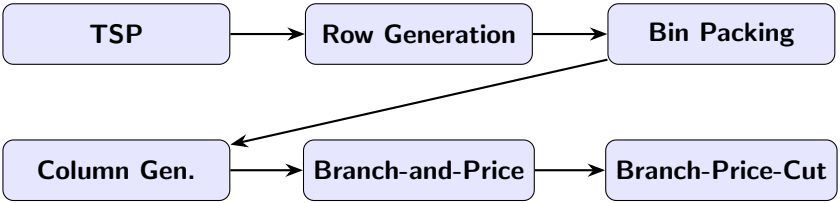
PySCIPOpt Workshop

Part 2: Advanced Techniques

PySCIPOpt Workshop

Roadmap

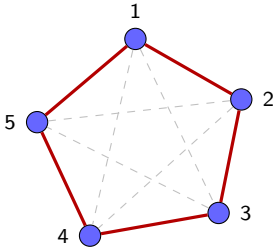


- Row generation: add **constraints** dynamically (TSP)
- Column generation: add **variables** dynamically (bin packing)
- Branch-and-price: column generation inside branch-and-bound
- Branch-price-and-cut: add cutting planes on top

Traveling Salesman Problem (TSP)

Given: n cities, pairwise distances d_{ij}
Find: shortest tour visiting each city exactly once, returning to start

- One of the most famous NP-hard problems
- Symmetric TSP: $d_{ij} = d_{ji}$ (undirected edges)
- We will compare two formulations:
  1. Compact MTZ (polynomial constraints, weak LP)
  2. Edge formulation + SECs (exponential constraints, strong LP)

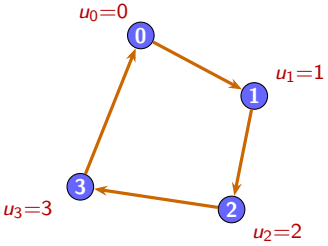


Optimal tour shown in red

MTZ Formulation

Variables: $x_{ij} \in \{0,1\}$ (directed edges), $u_i \in \mathbb{R}$ (positions)

$$
\begin{aligned}
\min \quad & \sum_{i \neq j} d_{ij}\, x_{ij} & & \\
\text{s.t.} \quad & \sum_{j \neq i} x_{ij} = 1 & \forall\, i \quad & \text{(leave once)} \\
& \sum_{i \neq j} x_{ij} = 1 & \forall\, j \quad & \text{(enter once)} \\
& u_i - u_j + n \cdot x_{ij} \leq n - 1 & \forall\, i, j \neq 0 \quad & \text{(MTZ)} \\
& x_{ij} \in \{0,1\} & &
\end{aligned}
$$



u_i values enforce a single directed tour

- $O(n^2)$ variables and constraints — no plugins needed
- LP relaxation is **very weak**

Reference: Compact MTZ

Already Implemented

Study the MTZ formulation as a baseline before implementing row generation.

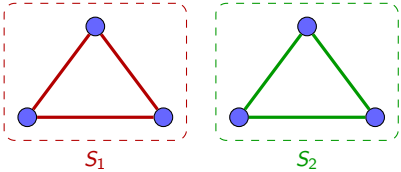
- File: `row_generation/compact_mtz.py`
- Run: `cd row_generation && python compact_mtz.py`

Edge Formulation with SECs

Variables: $x_e \in \{0,1\}$ for each undirected edge e

$$
\begin{aligned}
\min \quad & \sum_{e \in E} d_e \, x_e \\
\text{s.t.} \quad & \sum_{e \in \delta(v)} x_e = 2 && \forall\, v \in V \quad \text{(degree)} \\
& \sum_{e \in E(S)} x_e \leq |S| - 1 && \forall\, S \subset V \\
& 2 \leq |S| \leq n-1 && \text{(SECs)} \\
& x_e \in \{0,1\}
\end{aligned}
$$

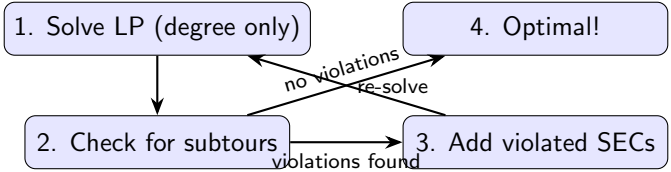
- **Exponentially many** SECs: $O(2^n)$ subsets
- **Much stronger** LP relaxation than MTZ



Subtour violation!

Degree = 2 but not connected

Row Generation Paradigm



- Start with a relaxed model (only degree constraints)
- Iteratively find and add violated constraints
- Converges to optimality without enumerating all 2^n SECs

Constraint Handler in SCIP

SCIP implements row generation via the **constraint handler** plugin.

Callback	Purpose
`conscheck`	Check if a candidate integer solution is feasible
`consenfolp`	Enforce constraints on LP solution; add cuts if violated
`conslock`	Declare variable locks (required; can be empty)

```
class SubtourElimination(Conshdlr):
    def conscheck(self, constraints, solution, ...):
        # Find subtours; return FEASIBLE or INFEASIBLE
    def consenfolp(self, constraints, ...):
        # Add SEC for each subtour; return CONSADDED
```

Exercises: Subtour Detection & Constraint Handler

Exercise 1: Subtour Detection

Implement `find_subtours()` — find connected components in the edge graph.

- File: `row_generation/subtour.py`
- Test: `python row_generation/test_subtour.py`

Exercise 2: Constraint Handler

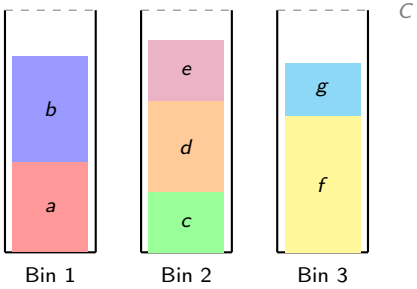
Implement `conscheck` and `consenfolp` callbacks.

- File: `row_generation/conshdlr_subtour.py`
- Test: `python row_generation/test_tsp.py`

Bin Packing Problem

Given: items with sizes s_i, bins with capacity C
Goal: pack all items using the fewest bins

- NP-hard; applications in logistics, scheduling, cloud computing
- We will explore two formulations:
  1. Compact (assignment variables) — symmetry, weak LP
  2. Extended (pattern variables) — column generation



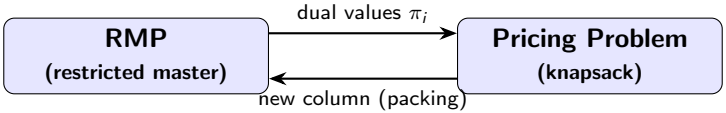
Compact Formulation

Variables: $x_{ib} \in \{0,1\}$ (item i in bin b), $y_b \in \{0,1\}$ (bin b used)

$$
\begin{aligned}
\min \quad & \sum_b y_b \\
\text{s.t.} \quad & \sum_b x_{ib} = 1 && \forall\, i \quad \text{(assign each item)} \\
& \sum_i s_i\, x_{ib} \leq C\, y_b && \forall\, b \quad \text{(capacity)} \\
& x_{ib}, y_b \in \{0,1\}
\end{aligned}
$$

Same solution, different indices!

Column Generation



1. Solve RMP with small set of columns
2. Get dual values π_i for each covering constraint
3. Solve pricing problem to find column with most negative reduced cost
4. If reduced cost ≥ 0: LP optimal. Else: add column, go to 1.

Extended Formulation

Let $\mathcal{P}$ = all feasible packings, $a_i^p = 1$ if item $i \in$ packing p:

$$
\begin{aligned}
\min \quad & \sum_{p \in \mathcal{P}} z_p & & \\
\text{s.t.} \quad & \sum_{p \in \mathcal{P}} a_i^p z_p = 1 & & \forall\, i \quad \text{(cover each item)} \\
& z_p \in \{0, 1\} & & \forall\, p
\end{aligned}
$$

- Exponentially many packings $\Rightarrow$ cannot enumerate
- Column generation generates only the useful ones
- **Stronger LP relaxation** than compact formulation

Pricing Problem = Knapsack

Reduced cost of a new packing a:

$$\bar{c} = 1 - \sum_i a_i \pi_i$$

Minimize reduced cost subject to capacity:

$$\begin{aligned} \min \quad & 1 - \sum_i a_i \, \pi_i \\ \text{s.t.} \quad & \sum_i s_i \, a_i \leq C \\ & a_i \in \{0, 1\} \end{aligned}$$

Equivalently: $\max \sum_i a_i \pi_i$ s.t. capacity $\Rightarrow$ **0–1 knapsack**!

When optimal reduced cost ≥ 0: no improving column exists.

Pricer Plugin in SCIP

Callback	Purpose
`pricerredcost`	Generate columns with negative reduced cost
`pricerfarkas`	Generate columns to restore feasibility

```
class KnapsackPricer(Pricer):
    def pricerredcost(self):
        duals = {i: self.model.getDualsolLinear(cons)
                 for i, cons in self.constraints.items()}
        red_cost, pattern = solve_knapsack(...)
        if red_cost < -1e-6:
            new_var = self.model.addVar(
                vtype="B", obj=1, pricedVar=True)
            # Add coefficients to covering constraints
        return {"result": SCIP_RESULT.SUCCESS}
```

Exercise: Knapsack Pricing

Task

Implement the 0–1 knapsack solver used as the pricing subproblem. Return `(optimal_value, list_of_selected_items)`.

- File: `branch_and_price/pricing_knapsack.py`
- Test: `cd branch_and_price && python test_pricing_knapsack.py`

Why Standard Branching Fails

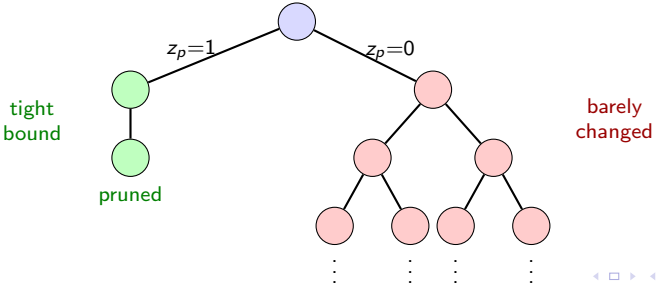
Standard variable branching on z_p:

$z_p = 1$

Force *one specific* packing out of exponentially many.
$\Rightarrow$ **Very restrictive**

$z_p = 0$

Forbid *one specific* packing out of exponentially many.
$\Rightarrow$ **Barely restrictive**



Ryan-Foster Branching

Branch on **pairs of items** (i, j) instead of individual variables:

Together

Items i and j must be in the **same** bin.

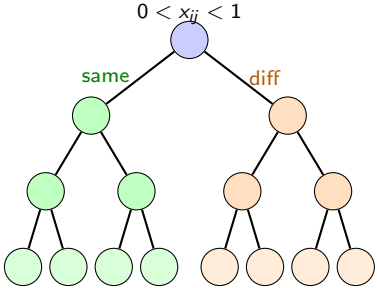
Apart

Items i and j must be in **different** bins.

How to find a pair:

- Compute implicit pair variable: $x_{ij} = \sum_{p:\, i \in p,\, j \in p} z_p$
- Choose pair where $0 < x_{ij} < 1$ (fractional)

$\Rightarrow$ Much more **balanced** search tree



Balanced tree

Exercises: Ryan-Foster Branching

Exercise: Fractional Pairs

Implement `all_fractional_pairs()` — find all item pairs with fractional implicit values.

- File: `branch_and_price/ryan_foster.py`
- Test: `cd branch_and_price && python test_fractional_pairs.py`

Exercise: Branching Decisions

Complete the child-node creation logic: inherit parent decisions, add new pair.

- File: `branch_and_price/ryan_foster.py`

Enforcing Branching in Pricing

The pricing problem must respect **all** accumulated branching decisions:

- **Together** (i, j): add constraint $a_i = a_j$ in knapsack
- **Apart** (i, j): add constraint $a_i + a_j \leq 1$ in knapsack

```
# Together constraints
for i, j in together:
    m.addCons(x[i] == x[j])

# Apart constraints
for i, j in apart:
    m.addCons(x[i] + x[j] <= 1)
```

Note: Apart does not forbid both items being *absent*.

Exercise: Constrained Pricing

Task

Implement `solve_knapsack_with_constraints()` — extend the knapsack solver with together/apart constraints from Ryan-Foster branching.

- File: `branch_and_price/pricing_knapsack.py`
- Test: `cd branch_and_price && python test_knapsack_with_constraints.py`

Integration Test

After all B&P exercises: `cd branch_and_price && python test_bnp.py`

Branch-Price-and-Cut

Strengthen branch-and-price by adding **cutting planes**:

Subset-row inequalities for set partitioning:
For any triple of items $S = \{i, j, k\}$, consider all packings covering ≥ 2 items in S. If their LP values sum > 1:

$$\sum_{p:\, |p \cap S| \geq 2} z_p \leq 1$$

Separator plugin in SCIP:

- `sepaexeclp`: search for violated inequalities at each LP solution
- Return `SEPARATED` if cut added, `DIDNOTFIND` otherwise

Exercise: Subset-Row Separator

Task

Implement the subset-row separator: enumerate triples of fractional LP variables and add cuts when their values sum exceeds 1.

- File: `separator/subset_row/subset_row.py`
- Test: `cd separator/subset_row && python test_subset_row.py`

Summary & Further Resources

What we covered:

- TSP: compact MTZ vs. row generation with SECs
- Bin packing: compact vs. extended formulation
- Column generation with knapsack pricing
- Branch-and-price with Ryan-Foster branching
- Branch-price-and-cut with subset-row inequalities

Bonus exercises (see `branch_and_price/README.md`):

- Dual stabilization, Lagrangian bound
- Better initialization, faster pricing
- Handling numerical issues

Resources:

- `https://www.scipopt.org/` — SCIP documentation
- `https://github.com/scipopt/PySCIPOpt` — PySCIPOpt